OOPS PROJECT PRESENTATION

FitMate: A Fitness and Workout Trainer

Department of Computer Science and Engineering | IIITV-ICD | November 2026

Team members

Team member	Roll number
Shivang Raj	20252501076
Bhumika Gupta	20252501095
Deepansh Singh	20252501019
Vir Sharma	20252501093

What is this project about?

FitMate is a fitness app built with object-oriented design. The user enters their details once, and the app:

- makes a 7-day workout plan
- records workouts and calories burned
- works out BMI and other health numbers
- sends daily reminders

IIITV & IIITV-ICD



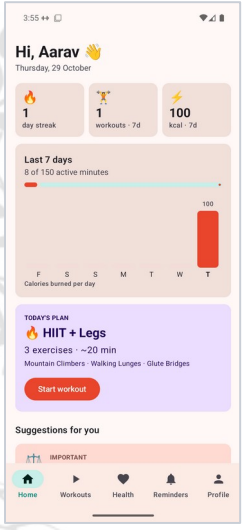


What Can FitMate Do?

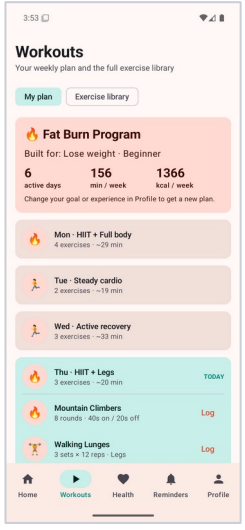
Main features

- **Profile:** your details and goal; bad values rejected
- **Weekly plan:** Day 1 to Day 7, based on the goal
- **Exercise library:** 23 exercises in 4 groups
- **Workout log:** calories are counted for you
- **Progress:** streak, 7-day chart, weekly minutes
- **Health:** BMI, BMR, daily calories, protein, water
- **Extras:** tips, YouTube tutorials, daily reminders

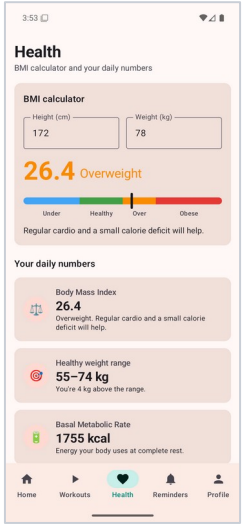
Main point: no login; all data stays on the phone.



Home



Workout plan

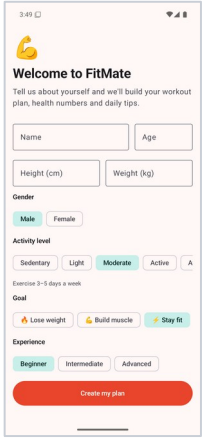


Health

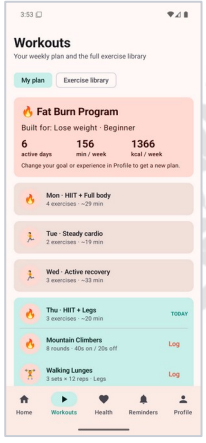
IIITV & IIITV-ICD

How Does the App Work?

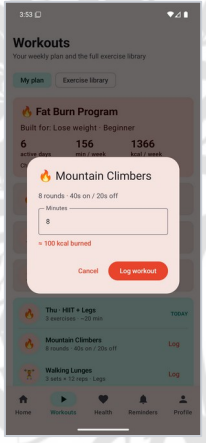
1. Create profile → 2. Weekly plan → 3. Log a workout → 4. Health and tips → 5. Reminders



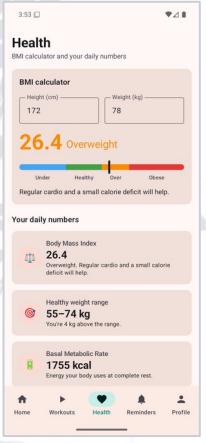
`UserProfile`



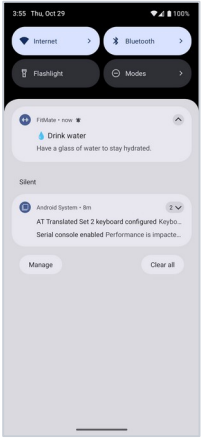
`PlanStrategy`



`WorkoutLog`



`HealthMetric`



`ReminderScheduler`

Main point: each step is handled by its own class (shown under each screen).

IIITV & IIITV-ICD

How Did We Build It?

Inside the Android app

Screens (UI): what the user sees and taps

↓

FitnessViewModel: holds the app's data

↓

Logic classes: plan, health, tips, progress

↓

Storage + alarms: saves data, sets reminders

Finding a tutorial video

User taps **Watch** → `TutorialApiClient`

↓

FastAPI server: `GET /tutorials/search`

↓

`YouTubeProvider` searches YouTube

↓

`rank_tutorials()` scores and sorts the videos

↓

Best video plays inside the app

Android	**iOS**	**Desktop**	**Backend**
Kotlin, Jetpack Compose	Swift, SwiftUI	Kotlin, Compose Desktop	Python, FastAPI

IIITV & IIITV-ICD

Which OOP Concepts Did We Use?

Concept	Where it is used in FitMate	Class
Class and object	Each workout the user logs is a new object	`WorkoutLog`
Encapsulation	The profile checks its own values before saving them	`UserProfile`
Inheritance	All exercise types reuse one parent class	`Exercise`
Polymorphism	Each exercise type counts calories its own way	`caloriesBurned()`
Abstraction	Screens use a health metric without knowing its formula	`HealthMetric`
Interface	Every plan class has the same buildPlan() method	`PlanStrategy`
Generics	One storage class saves workouts and reminders	`Repository<T>`
Composition	A plan has 7 sessions; each session has exercises	`WorkoutPlan`

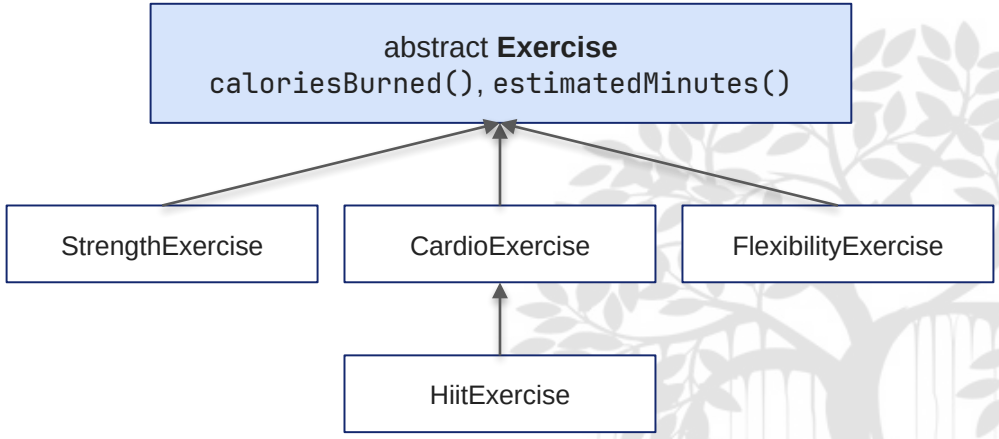
The code

```
// Encapsulation: the setter checks the value
var age: Int = 0
  set(value) {
    require(value in MIN_AGE..MAX_AGE) { ... }
    field = value
  }
```

```
// Abstraction: screens see only this
interface HealthMetric {
  fun calculate(profile: UserProfile): Double
  fun interpret(profile: UserProfile): String
}
class BmiMetric : HealthMetric { ... }
```

```
// Generics: one interface, many types
interface Repository<T : Identifiable> {
  fun getAll(): List<T>
  fun save(item: T)
  fun delete(id: String)
}
```

Inheritance and Polymorphism: Exercises



What the diagram shows

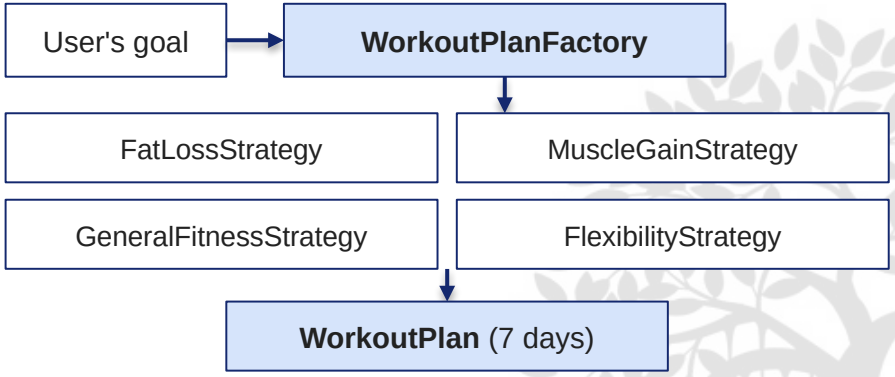
- `Exercise` is abstract and holds the shared data.
- Each type gives its own time and calorie formula.
- `HiitExercise` extends `CardioExercise`: multi-level inheritance.
- The app treats every type as an `Exercise`.

The code

```
// HiitExercise: reuse the parent's calorie formula, then add a 15% afterburn bonus
override fun caloriesBurned(weightKg: Double, minutes: Int): Double =
    super.caloriesBurned(weightKg, minutes) * AFTERBURN_BONUS
```

IIITV & IIITV-ICD

Building the Weekly Plan: Strategy and Factory




```
// The factory picks the right plan class for the goal
fun strategyFor(goal: FitnessGoal): PlanStrategy =
  when (goal) {
    LOSE_WEIGHT → FatLossStrategy()
    BUILD_MUSCLE → MuscleGainStrategy()
    STAY_FIT → GeneralFitnessStrategy()
    IMPROVE_FLEXIBILITY → FlexibilityStrategy()
  }
```

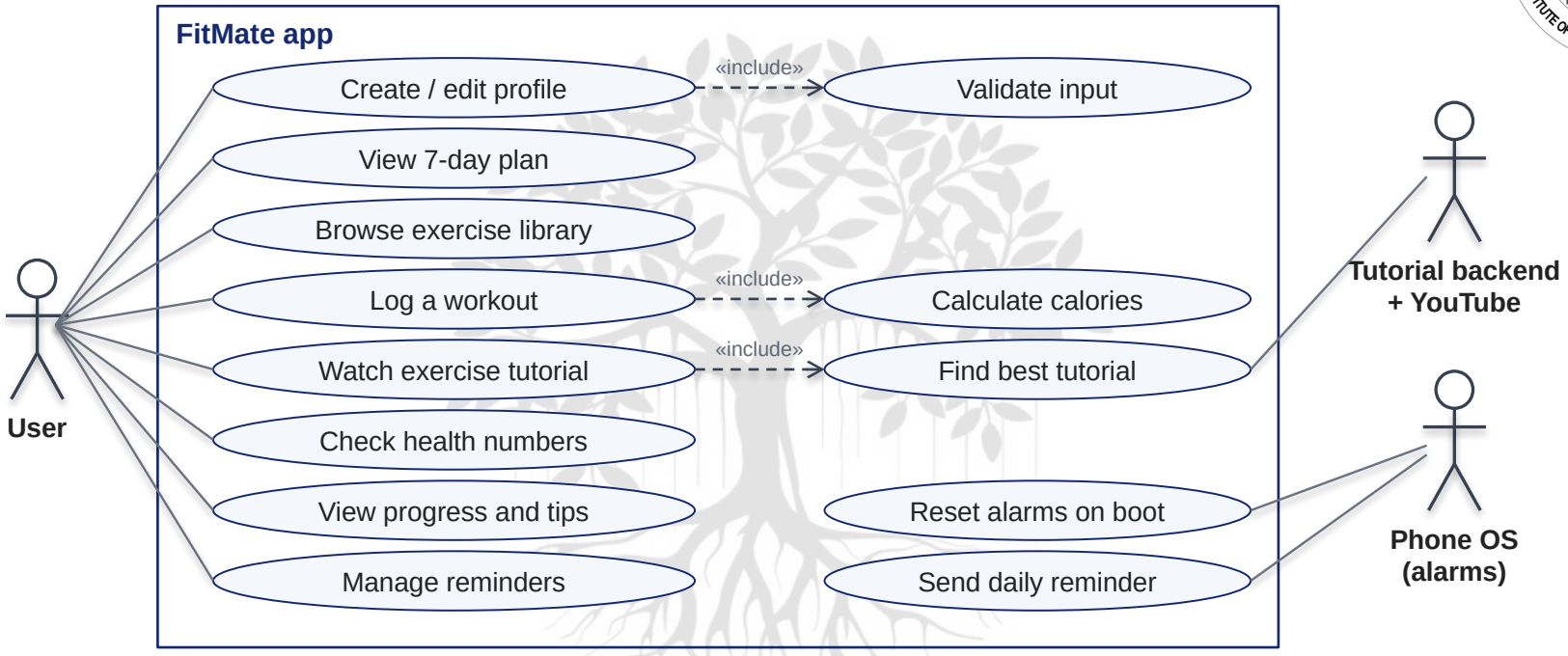
Design patterns we used

Pattern	In FitMate
Strategy	One plan class per goal
Factory	`WorkoutPlanFactory`
Singleton	`ExerciseLibrary` (one copy)
Repository	`Repository<T>` hides storage
Observer	Screens redraw when data changes
Dependency injection	`AppContainer` hands out objects

Each goal's plan lives in its own class.

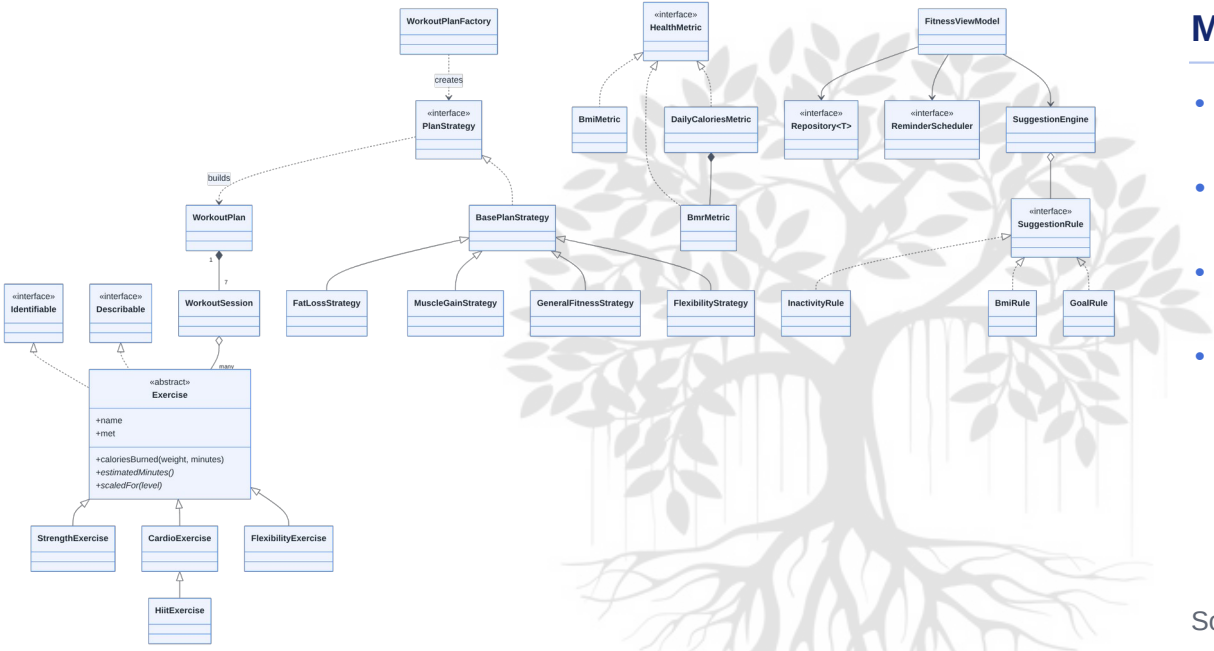
IIITV & IIITV-ICD

Use Case Diagram



IIITV & IIITV-ICD

Class Diagram



Main classes

- `WorkoutPlan` is made of 7 `WorkoutSessions`.
- Four plan classes extend `BasePlanStrategy`.
- `HealthMetric` and `SuggestionRule` are interfaces.
- `FitnessViewModel` uses the repository, the scheduler and the suggestion engine.

Source: FitMate project repository

IIITV & IIITV-ICD

Conclusion: What Did We Learn?

What we built

1. FitMate for Android, iOS and desktop, with a Python backend.
2. Every main OOP concept is used in a real feature.
3. Strategy and Factory made the plan code easy to extend.
4. 14 Android tests and 5 backend tests check the core logic.

What we learned

- Plan the classes before writing the screens.
- Small classes with one job are easier to test.
- Interfaces let us change storage or alarms without touching the screens.

Thank you!

IIITV & IIITV-ICD